

AI Usage Note

Project: Schema Aware Test Data Generator

Team: T C Varalakshmi, V. Sai Ranga Prasad, S. Pranathi

What AI Helped With

During the development of this project, we used Google Gemini (via API) as our AI layer. Here is where it genuinely helped us:

- When we uploaded a Banking schema, Gemini correctly identified it as a "Banking / Finance System" without us hardcoding that logic. That saved us writing a lot of manual if-else conditions.
 - The semantic column mapping was surprisingly useful. Columns like emp_sal, ph_no, and cust_nm were decoded into readable labels automatically. This helps testers understand what each column actually means.
 - The data quality review gave us a score out of 100 after looking at 10 sample rows per table. It caught cases where our generator was producing sentence-like text in columns that should have had short values.
 - We used AI assistance (Claude) during development to debug issues in our generator.py where account_number and account_type columns were getting random sentences instead of proper values like ACC-000001 and Savings.
-

What AI Got Wrong

We want to be honest about where AI did not perform well:

- **Gemini returned inconsistent JSON** — sometimes it added extra text before the JSON object, which caused our parser to crash. We had to add a `raw.find("{")` workaround to extract just the JSON part.
- **Old Gemini model (text-bison-001) did not work** — we initially used the wrong model and endpoint URL from an older version of the API. The app was silently failing without any error. We had to manually update it to gemini-1.5-flash with the correct v1beta endpoint.
- **Semantic mapping was sometimes too generic** — for columns like notes or description, Gemini just returned the same word back without any useful interpretation.
- **Data quality score was not always reliable** — for simple schemas with 2-3 tables, Gemini sometimes gave a score of 100 even when some columns had borderline values. We treated the score as a guidance indicator, not ground truth.

Best Prompts We Used :

1. Schema Understanding Prompt

You are a database expert. Analyze this SQL schema and return ONLY a JSON object with these keys: business_domain, schema_description, system_type, recommendations (list of 3 strings). No extra text.

Tables and columns:

accounts: account_id, account_number, account_type, balance

clients: client_id, client_name, client_email, created_at

This worked best because we said "ONLY JSON" and "No extra text" — without that, Gemini added paragraphs before the JSON.

2. Data Quality Review Prompt

Review these generated database rows. For each table check if values match column intent. Give an overall score (0-100) and 3 short recommendations.

Return ONLY JSON:

```
{"data_quality_score": 85, "validation_summary": {}, "recommendations": []}
```

Giving an example of the exact JSON format in the prompt made the output far more consistent.

3. Semantic Column Mapping Prompt

You are a database expert. Decode these column names from table 'employees'.

Columns: [emp_id, emp_sal, ph_no, doj, dept_nm]

Return ONLY a JSON object like {"column_name": "human readable label"}.

Examples: cust_nm → customer_name, emp_sal → employee_salary

Including examples directly in the prompt (few-shot prompting) gave much better results than asking without examples.

Our Honest Takeaway :

AI was most useful for **understanding and classification tasks** — things that need human-like reading of context. It was not reliable for **structured data generation** — that is why our core generation engine uses Python + Faker, not AI. We used AI only where it genuinely added value, and kept a fallback so the app works even without an API key.